



SMIT SIKKIM
MANIPAL
UNIVERSITY
SIKKIM MANIPAL INSTITUTE OF TECHNOLOGY

Project Report

On:

Distributed Real-Time Observability and Log Intelligence Platform

Major Project Phase-II Progress Presentation

Submitted in partial fulfilment of the award of the degree of Bachelor of
Technology in Computer Science & Engineering.

By:

Vibhu Garg (202200062)

Under the Guidance of:

Mr. Shubham Ranjan

Associate Professor II

Department of Computer Science and Engineering, SMIT

Department of Computer Science & Engineering
Sikkim Manipal Institute of Technology
(A constituent college of Sikkim Manipal University)
Majitar, Rangpo, East Sikkim – 737136

TABLE OF CONTENTS

S. NO.	TITLE	PAGE NO.
<i>1.</i>	Abstract	3
<i>2.</i>	Introduction	4
<i>3.</i>	Literature Survey	5-6
<i>4.</i>	Problem Definition	7
<i>5.</i>	Solution Strategy	8
<i>6.</i>	Methodology	9-10
<i>7.</i>	Work Done and Outcome	11-12
<i>8.</i>	Future Scope	13
<i>9.</i>	Gantt Chart	14
<i>10.</i>	References	15

ABSTRACT

Modern distributed applications generate massive volumes of logs and system events during their execution. These logs contain critical information regarding system performance, failures, and operational behavior. However, managing and analyzing large-scale log data in real time remains a significant challenge for modern software systems. Traditional logging systems often struggle to process high-velocity log streams efficiently and provide meaningful insights for system monitoring and debugging.

This project proposes a **Distributed Real-Time Observability and Log Intelligence Platform** designed to ingest, process, analyze, and visualize log data in real time using a scalable distributed pipeline architecture. The platform enables applications to send log data through a high-speed ingestion gateway, where logs are validated and forwarded to a buffering layer using Redis Streams. This buffering mechanism ensures reliable log delivery and prevents system overload during sudden traffic spikes

The system incorporates worker services responsible for processing log data asynchronously. These workers normalize logs, perform trace ID correlation, and apply statistical anomaly detection techniques to identify abnormal system behavior. Processed logs are then stored in **MongoDB time-series collections**, which provide efficient storage and retrieval for time-based log data.

A real-time monitoring dashboard built using **React and WebSocket technologies** provides live visualization of logs, system metrics, and alerts. The dashboard allows engineers to track system health, identify error spikes, and investigate distributed request traces quickly.

Overall, the proposed platform aims to enhance system observability, improve failure detection, and reduce debugging time by providing a scalable and intelligent log monitoring solution suitable for modern distributed systems.

INTRODUCTION

Modern software systems increasingly rely on distributed architectures such as microservices and cloud-based infrastructures. These systems generate a continuous stream of logs and system events that provide valuable insights into application performance, operational status, and potential system failures. Logs play a critical role in debugging, monitoring system behavior, and ensuring the reliability of large-scale applications.

However, managing and analyzing log data from distributed environments presents several challenges. Applications often generate thousands of logs per second, making it difficult to store, process, and analyze this data efficiently. Traditional logging systems typically store logs without providing advanced analytics or real-time monitoring capabilities, making it harder for engineers to detect failures or abnormal patterns quickly.

Another challenge arises from the distributed nature of modern applications. A single user request may pass through multiple services, each generating its own logs. Without proper correlation mechanisms, tracing the root cause of failures across multiple services becomes complex and time-consuming.

Observability platforms aim to address these challenges by collecting logs, metrics, and traces to provide comprehensive insights into system behavior. Modern observability systems use distributed pipelines, message queues, and analytics engines to process large volumes of telemetry data efficiently.

The proposed **Distributed Real-Time Observability and Log Intelligence Platform** aims to build a scalable solution for log ingestion, processing, and analysis. The platform collects logs from multiple applications, processes them through a distributed pipeline, and performs anomaly detection to identify abnormal system behavior. Through real-time visualization and alerting capabilities, the system enables engineers to monitor system health and respond quickly to potential failures.

By combining distributed processing, log analytics, and real-time visualization, the platform provides a comprehensive observability solution for monitoring modern distributed systems.

LITERATURE SURVEY

SL No	Author	Paper Title and Publication Details	Findings	Research Gaps
1.	S. He, P. He, Z. Chen, T. Yang, Y. Su and M. R. Lyu	“A Survey on Automated Log Analysis for Reliability Engineering,” ACM Computing Surveys, 2021	The study presents various techniques for automated log analysis used to detect system failures and anomalies in large-scale distributed systems. It highlights how machine learning and pattern mining can be applied to analyze system logs for reliability engineering.	Most approaches focus on offline log analysis and do not address scalable real-time log processing systems capable of handling high-speed log streams.
2.	X. Han, S. Yuan and M. Trabelsi.	“LogGPT: Log Anomaly Detection via GPT,” arXiv preprint, 2023	This research introduces the use of large language models for detecting anomalies in system logs by learning semantic patterns within log sequences. The approach improves anomaly detection accuracy in complex distributed systems.	The research focuses primarily on anomaly detection algorithms and does not address the design of distributed log ingestion pipelines and monitoring dashboards.

3.	Y. Wang, M. Mäntylä, J. Nyyssölä, K. Ping and L. Wang	“Cross-System Software Log-based Anomaly Detection Using Meta-Learning,” arXiv preprint, 2024	The paper proposes a meta-learning framework capable of detecting anomalies across different software systems using log data. It improves detection performance in heterogeneous environments.	The study focuses on machine learning models rather than providing a full observability platform capable of ingesting, processing, and visualizing log data in real time.
----	---	---	--	---

PROBLEM DEFINITION

Modern distributed applications such as cloud platforms and microservice-based systems generate large volumes of log data continuously. These logs contain valuable information about system operations, performance metrics, and potential errors. However, managing and analyzing such high volumes of log data remains a significant challenge for software engineers and system administrators.

Traditional logging systems often store logs in centralized databases or files without providing advanced mechanisms for real-time processing and analysis. As the number of services and users increases, the volume of log data grows rapidly, making it difficult to identify system failures or abnormal behavior in a timely manner.

Another major challenge is the distributed nature of modern applications. A single user request may travel across multiple services, each producing its own logs. Without proper correlation mechanisms, identifying the root cause of failures across multiple services becomes complex and time-consuming.

Furthermore, many existing monitoring tools are expensive, resource-intensive, or difficult to deploy in smaller environments. Organizations often require lightweight solutions that can provide real-time insights without significant infrastructure overhead.

Therefore, there is a need for a scalable and efficient system capable of collecting, processing, analyzing, and visualizing log data in real time. Such a system should support distributed log ingestion, anomaly detection, and real-time monitoring to improve system observability and reliability.

SOLUTION STRATEGY

To overcome the challenges of large-scale log monitoring and analysis in distributed systems, the proposed platform implements a scalable observability pipeline that collects, processes, and visualizes log data in real time. The system is composed of several key components that work together to ensure efficient log ingestion, processing, storage, and monitoring.

1. Log Ingestion Gateway

The system begins with a high-performance ingestion gateway responsible for receiving log data from multiple applications. The gateway validates incoming log payloads and generates unique trace identifiers to track requests across services. After validation, logs are immediately forwarded to the buffering layer to ensure low-latency ingestion.

2. Buffering Layer (Redis Streams)

Redis Streams is used as a buffering and message queue system to handle large volumes of log data. Incoming logs are temporarily stored in Redis Streams and distributed to worker services using consumer groups. This mechanism helps manage traffic spikes and ensures reliable log delivery.

3. Worker Processing System

Worker services operate as background processors that read logs from Redis Streams and perform tasks such as log normalization, classification of log levels, and anomaly detection. This distributed processing approach ensures efficient handling of large log streams.

4. Log Storage

Processed logs are stored in MongoDB using time-series collections. This database structure allows efficient storage and retrieval of time-based log data while enabling engineers to analyze historical logs and system behavior over time.

5. Monitoring and Visualization

The system includes a real-time monitoring dashboard developed using React and WebSocket technologies. The dashboard displays live log streams, system metrics, and alerts, enabling engineers to monitor system health and quickly identify abnormal behavior.

METHODOLOGY

- **Requirement Analysis**
 - Studied challenges of traditional logging systems such as high latency, poor scalability, and delayed failure detection.
 - Identified need for real-time ingestion, buffering, storage, and monitoring.
- **System Architecture Design**
 - Designed a distributed pipeline with separate layers for ingestion, queueing, processing, storage, and visualization.
 - Planned modular architecture for easy scaling and maintenance.
- **Backend API Development**
 - Developed REST API using **Node.js** and **Fastify**.
 - Created endpoint to receive logs from applications using HTTP POST requests.
- **Schema Validation**
 - Implemented **Zod** validation for incoming log payloads.
 - Ensured required fields such as service, message, level, and traceId are present and correctly formatted.
- **Queue Integration using Redis Streams**
 - Integrated **Redis Streams** as a buffering layer.
 - Queued validated logs before processing.
 - Enabled handling of traffic spikes without slowing API response.
- **Worker Service Development**
 - Built separate worker process to consume logs asynchronously.
 - Used Redis consumer groups for reliable message processing.
 - Sent acknowledgment after successful processing.
- **Database Integration**
 - Connected **MongoDB** for persistent storage.
 - Stored processed logs with fields such as timestamp, traceId, and processedAt.
- **Testing and Validation**
 - Performed functional testing using **Postman**.
 - Verified stored logs in MongoDB.
 - Conducted load testing using **Artillery** with 500 requests.
- **Future Enhancement Planning**
 - Planned React dashboard for live monitoring.
 - Planned WebSocket updates, anomaly detection, and multi-worker scaling.

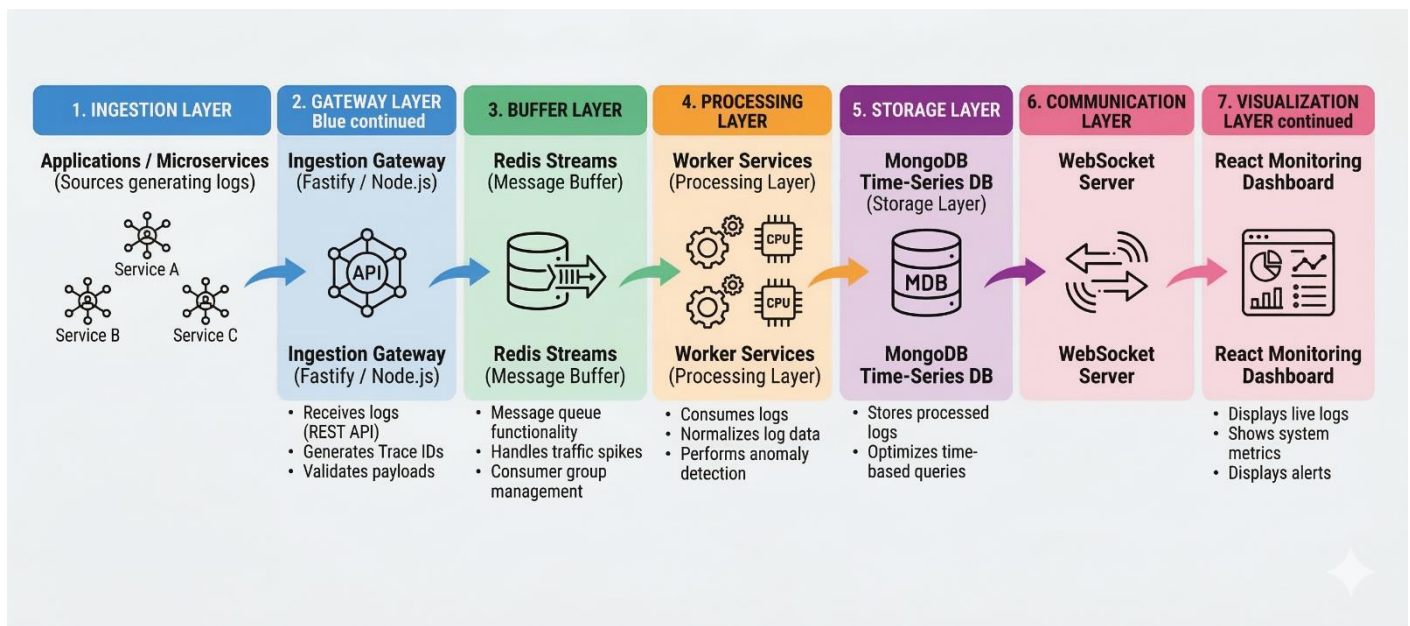


Fig 1: Workflow Diagram

Work Done

- **Backend API Development Completed**

A high-performance backend server was developed using Node.js and Fastify. The server provides REST API endpoints through which applications can send log data in real time. Fastify was chosen because of its speed and lightweight architecture.

- **Structured Log Validation Implemented**

Zod schema validation was integrated to verify incoming log payloads. Required fields such as service name, message, severity level, and trace ID are checked before logs are accepted, ensuring consistency and preventing invalid data entry.

- **Redis Streams Integration Completed**

Redis Streams was configured as the queueing and buffering layer. Instead of directly storing logs in the database, validated logs are first pushed into the stream. This helps the system manage sudden traffic spikes efficiently.

- **Worker Service Implemented**

A separate worker process was developed to consume logs asynchronously from Redis Streams. Consumer groups were used for reliable processing, and logs are acknowledged only after successful completion.

- **MongoDB Storage Integrated**

MongoDB was connected as the persistent storage layer. Processed logs are stored with fields such as service name, message, level, timestamp, trace ID, and processing time for future querying and analysis.

- **Docker Environment Configured**

Redis and MongoDB services were containerized using Docker Compose. This simplified setup, deployment, and portability of the project environment.

- **Functional Testing Completed**

The backend API was tested using Postman to verify request handling, validation, and correct responses. Stored logs were also verified in Redis Streams and MongoDB.

- **Load Testing Performed**

System performance under traffic load was tested using Artillery by generating multiple concurrent requests.

Results Obtained

- Total Requests Sent: 500
- Successful Responses Received: 500
- HTTP 200 Success Rate: 100%
- Average Response Time: 6.9 ms
- Median Response Time: 3 ms
- Failed Requests: 0
- Stable Throughput Achieved: 50 requests per second

Output

The implemented system successfully demonstrated an end-to-end asynchronous log processing pipeline. Logs were received, validated, queued, processed, and stored efficiently. Test results confirmed that the architecture can handle concurrent traffic with low latency and high reliability.

Future Scope

The current implementation successfully establishes the backend foundation of a real-time log intelligence platform. However, several advanced features can be added in future phases to enhance the system's capabilities, scalability, and usability.

- **React-based Monitoring Dashboard**
A user-friendly dashboard can be developed using **React.js** to provide live visualization of logs, system metrics, and service health. This will help engineers monitor system behavior more effectively.
- **Real-Time Updates using WebSockets**
Integration of **WebSocket** technology can enable instant log updates on the dashboard without requiring manual page refresh, improving real-time observability.
- **Trace ID Correlation View**
A dedicated tracing module can be added to track a single request across multiple microservices using the trace ID. This will simplify debugging and root cause analysis in distributed systems.
- **Anomaly Detection Module**
Machine learning or statistical models can be integrated to automatically detect unusual log patterns, sudden error spikes, or suspicious activities.
- **Alert and Notification System**
The platform can be extended to send alerts through email, SMS, or messaging tools such as Slack whenever critical errors or abnormal conditions are detected.
- **Multi-Worker Horizontal Scaling**
Multiple worker instances can be deployed to process logs in parallel, increasing throughput and improving system performance under heavy traffic.
- **Cloud Deployment**
The platform can be deployed on cloud environments such as AWS, Azure, or Google Cloud for high availability and global accessibility.
- **Security Enhancements**
Authentication, role-based access control, encrypted communication, and secure log transmission can be implemented for enterprise-level usage.
- **Advanced Analytics and Reporting**
Future versions may include trend analysis, error summaries, downloadable reports, and historical performance dashboards.

GANTT CHART

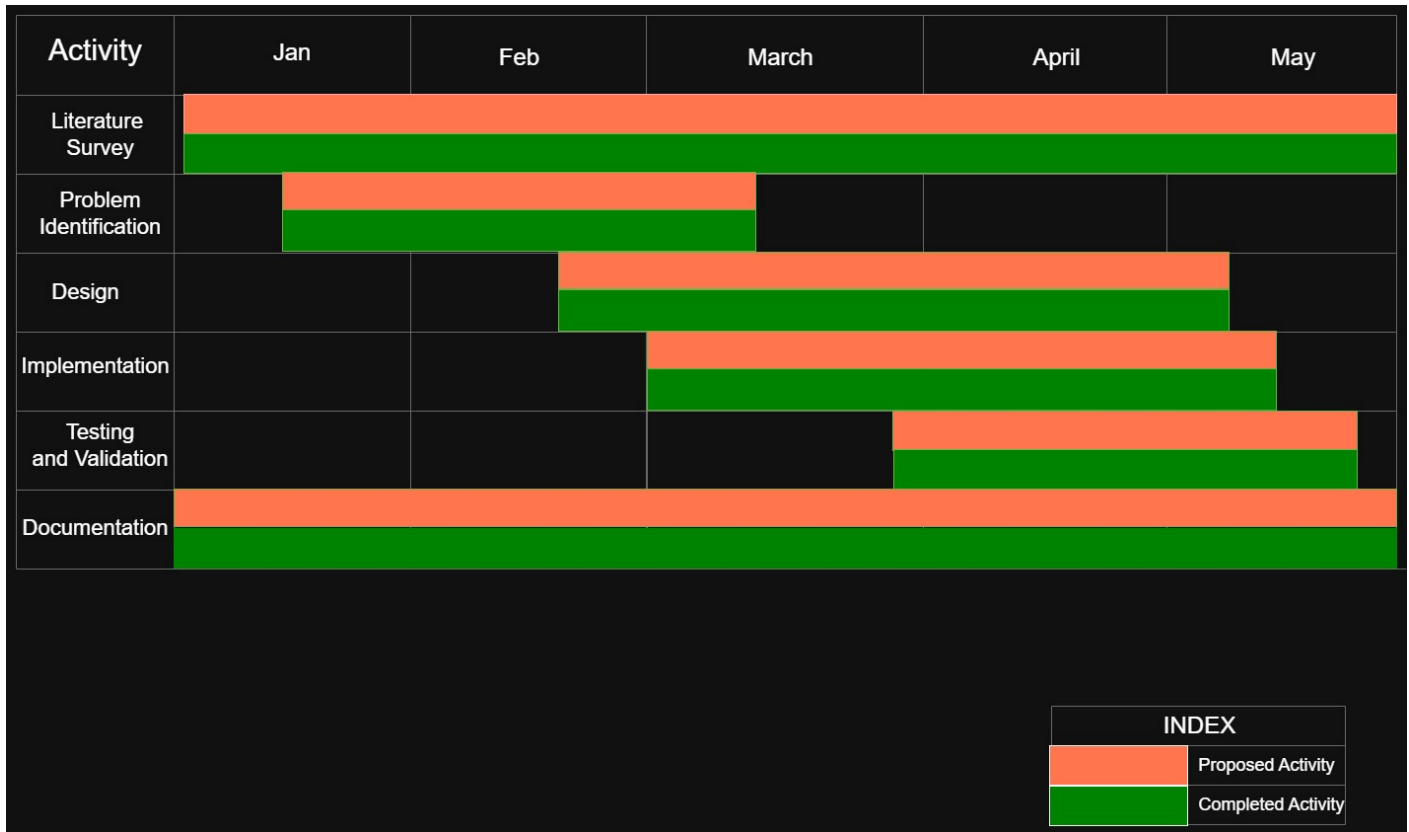


Fig 2: Gantt Chart

REFERENCES

- [1] Bhole, Varsha, et al. "Automating Cross-Platform Task Management with TaskFlow." [1] S. He, P. He, Z. Chen, T. Yang, Y. Su and M. R. Lyu, "A Survey on Automated Log Analysis for Reliability Engineering," *ACM Computing Surveys*, vol. 54, no. 6, 2021. <https://arxiv.org/abs/2009.07237>
- [2] [2] X. Han, S. Yuan and M. Trabelsi, "LogGPT: Log Anomaly Detection via GPT," *arXiv preprint*, 2023. <https://arxiv.org/abs/2309.14482>
- [3] [3] Y. Wang, M. Mäntylä, J. Nyssölä, K. Ping and L. Wang, "Cross-System Software Log-based Anomaly Detection Using Meta-Learning," *arXiv preprint*, 2024. <https://arxiv.org/abs/2412.15445>
- [4] [4] W. Xu, L. Huang, A. Fox, D. Patterson and M. Jordan, "Detecting Large-Scale System Problems by Mining Console Logs," *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2009. <https://people.eecs.berkeley.edu/~jordan/papers/xu-etal-sosp09.pdf>
- [5] [5] Grafana Labs, "Grafana Documentation – Visualization and Observability Platform," <https://grafana.com/docs/grafana/latest/>
- [6] [6] Redis Ltd., "Redis Streams Documentation – Data Stream Processing," <https://redis.io/docs/data-types/streams/>
- [7]